UNIT IV

Files & Object-Oriented Programming

Text & binary files · pickle · CSV · os and os.path · Classes, objects, constructors · Encapsulation · Inheritance · Polymorphism · Sample Programs 18–23

➤ What you will learn in this unit

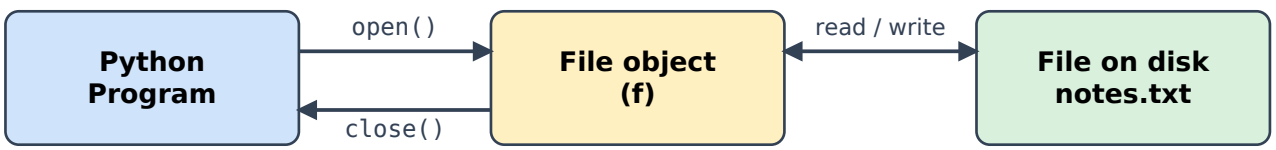
- How Python stores data permanently — reading and writing text, binary and CSV files.
- How to save whole Python objects with the **pickle** module and work with folders using **os** / **os.path**.
- The OOP way of thinking: classes, objects, constructors, attributes, encapsulation, inheritance and polymorphism.
- Solutions of the sample programs **18 to 23** with real output.

PART A — FILES

1. Why do we need files? Types of files

Variables live in RAM, so everything is lost when the program ends. A **file** is a named place on the disk where data is stored **permanently**. Python treats every file as a stream of data that can be read or written.

	Text file	Binary file
Content	Human-readable characters (letters, digits, symbols)	Raw bytes (0s and 1s) — not readable in a text editor
Examples	.txt, .py, .csv, .html	.jpg, .png, .mp3, .pdf, .exe, .pkl
Line ending	Each line ends with newline character (**\n**)	No concept of lines
Open modes	`"r" "w" "a"`	`"rb" "wb" "ab"`
Handled with	read(), readline(), write()	read(), write() with **bytes**; pickle module



Every file operation: OPEN → PROCESS (read / write) → CLOSE

2. Opening and closing a file

Syntax: `file_object = open(filename, mode)`. The **mode** tells Python what you want to do with the file.

Mode	Meaning	If file does not exist	Old content
`r`	Read only (default)	Error (FileNotFoundError)	Kept
`w`	Write only	Creates a new file	**Erased** (overwritten)
`a`	Append — add at the end	Creates a new file	Kept
`r+`	Read and write	Error	Kept
`w+`	Write and read	Creates a new file	**Erased**
`a+`	Append and read	Creates a new file	Kept
`rb wb ab`	Same as r / w / a but in **binary** mode	As above	As above

Always close a file after use so that data is saved properly. The safest way is the **with** statement — it closes the file **automatically**, even if an error occurs.

```
f = open("notes.txt", "w")      # open
f.write("Hello Python")         # process
f.close()                       # close

with open("notes.txt", "r") as f:   # recommended: closes automatically
    print(f.read())
    print(f.name, f.mode, f.closed) # file attributes
```

Output

```
Hello Python
notes.txt r False
```

3. Reading and writing text data

Reading methods

Method	What it does
`f.read()`	Reads the **entire** file as one string. *read(n)* reads only n characters.
`f.readline()`	Reads **one line** (including the newline) each time it is called.
`f.readlines()`	Reads all lines and returns them as a **list** of strings.
`for line in f:`	Reads the file line by line — best for big files (memory-friendly).

Writing methods

Method	What it does
`f.write(s)`	Writes the string *s*. Does **not** add a newline automatically. Returns number of characters written.
`f.writelines(list)`	Writes every string of a list (add "\n" yourself).
`f.tell()`	Returns the current position of the file pointer.
`f.seek(n)`	Moves the file pointer to byte number n (e.g. seek(0) goes back to start).

```
with open("data.txt", "w") as f:
    f.write("Line 1\n")
    f.writelines(["Line 2\n", "Line 3\n"])

with open("data.txt", "a") as f:        # append mode
    f.write("Line 4\n")

with open("data.txt") as f:
    print(f.readline().strip())         # first line
    print(f.readlines())                # remaining lines as list
```

Output

```
Line 1
['Line 2\n', 'Line 3\n', 'Line 4\n']
```

! Careful!

- Mode **"w"** destroys the old content of an existing file. Use **"a"** when you only want to add data.
- After *read()*, the pointer is at the end — a second read() returns an empty string unless you call *seek(0)*.

4. Reading and writing binary files

Binary files store bytes. Open them with **"rb"** / **"wb"**; data read is of type **bytes**, and data to write must also be bytes. A typical use is copying images, audio or any non-text file.

```
with open("photo.jpg", "rb") as src, open("copy.jpg", "wb") as dst:
    data = src.read()           # bytes
    dst.write(data)             # writes the same bytes

b = bytes([72, 105])            # bytes object  -> b'Hi'
print(b, b.decode())
```

Output

```
b'Hi' Hi
```

5. Pickle module — saving whole objects

Pickling (serialisation) converts a Python object (list, dictionary, class object…) into a stream of bytes so it can be stored in a file; **unpickling** converts it back to the object.




```
import pickle
student = {"name": "Asha", "marks": [78, 85, 90]}

with open("student.pkl", "wb") as f:     # binary write
    pickle.dump(student, f)

with open("student.pkl", "rb") as f:     # binary read
    data = pickle.load(f)
print(data)
```

Output

```
{'name': 'Asha', 'marks': [78, 85, 90]}
```

! Careful!

- Only unpickle files from **trusted** sources — a malicious pickle file can run harmful code.

6. Reading and writing CSV files

CSV (Comma Separated Values) stores table-like data: one row per line, values separated by commas. Python's **csv** module handles it.

Function / class	Purpose
`csv.reader(f)`	Reads rows — each row is a **list** of strings.
`csv.writer(f)`	Has writerow(list) and writerows(list of lists) to write data.
`csv.DictReader(f)`	Reads each row as a **dictionary** (header row gives the keys).
`csv.DictWriter(f, fieldnames)`	Writes dictionaries as rows (writeheader() writes the header).

```
import csv
with open("marks.csv", "w", newline="") as f:      # newline="" avoids blank lines
    w = csv.writer(f)
    w.writerow(["Name", "Marks"])
    w.writerows([["Asha", 90], ["Ravi", 75]])

with open("marks.csv") as f:
    for row in csv.reader(f):
        print(row)
```

Output

```
['Name', 'Marks']
['Asha', '90']
['Ravi', '75']
```

7. The os and os.path modules

The **os** module lets a program talk to the operating system — create, delete, rename files and folders. **os.path** works with file paths.

os function	Use	os.path function	Use
`os.getcwd()`	Current working directory	`path.exists(p)`	Does the path exist?
`os.listdir(p)`	List files and folders	`path.isfile(p)`	Is it a file?
`os.mkdir(name)`	Create one folder	`path.isdir(p)`	Is it a folder?
`os.makedirs(path)`	Create nested folders	`path.join(a, b)`	Join path parts safely
`os.rmdir(name)`	Remove empty folder	`path.basename(p)`	File name part of a path
`os.remove(file)`	Delete a file	`path.dirname(p)`	Folder part of a path
`os.rename(old, new)`	Rename file / folder	`path.splitext(p)`	Split name and extension
`os.chdir(path)`	Change directory	`path.getsize(p)`	File size in bytes

```
import os
```

```
print(os.path.exists("data.txt"))
p = os.path.join("reports", "marks.csv")
print(p, os.path.basename(p), os.path.splitext(p))
```

Output

```
False
reports/marks.csv marks.csv ('reports/marks', '.csv')
```

★ Study Tip

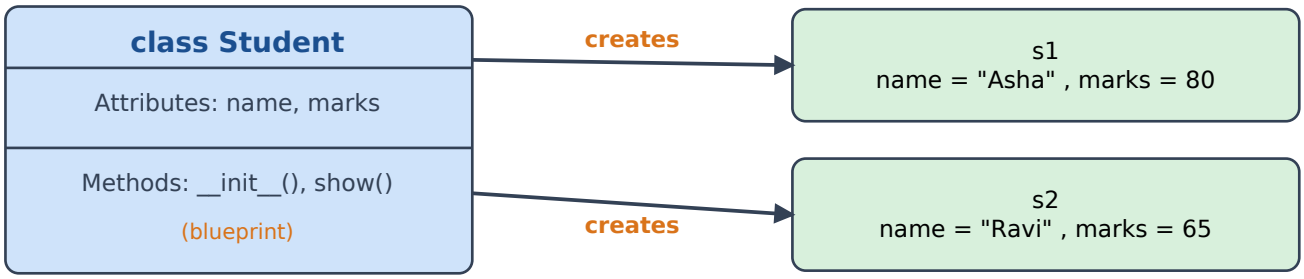
- File handling exam answers score well when you remember the cycle **open → read/write → close** and the table of **modes**.
- Draw the small flow diagram of pickle / file handling — it earns easy marks.

PART B — OBJECT-ORIENTED PROGRAMMING

8. Classes and objects

Object-Oriented Programming (OOP) organises a program around **objects** — things that hold **data** (attributes) and the **functions** that work on that data (methods). Real world: a *Student* has a name and marks (data) and can show details (action).

- **Class** — a blueprint / template (e.g. "Student"). It creates no memory for data by itself.
- **Object** — a real instance built from the class (e.g. Asha, Ravi). Each object has its own copy of the data.
- **Attribute** — variable inside a class/object. **Method** — function inside a class.
- **self** — refers to the **current object**; it must be the first parameter of every instance method.



Creating a class and objects

```
class Student:                      # creating a class
    def show(self):                 # method
        print("I am a student")

s1 = Student()                      # creating an object
s1.show()                           # calling a method with the object
```

Output

```
I am a student
```

9. Constructor method and multiple objects

The **constructor** `__init__(self, ...)` is a special method that runs **automatically** when an object is created. It is used to give initial values to the attributes. (Python has one constructor per class — default values can replace "constructor overloading".)

```
class Student:
    def __init__(self, name, marks):     # constructor
        self.name = name                 # instance attributes
        self.marks = marks

    def show(self):
        print(self.name, "scored", self.marks)

s1 = Student("Asha", 80)     # multiple objects of the same class
s2 = Student("Ravi", 65)
s1.show()
s2.show()
print(s1 is s2)              # two different objects
```

Output

```
Asha scored 80
Ravi scored 65
False
```

10. Class attributes vs instance (data) attributes

	Instance attribute	Class attribute
Defined	Inside `__init__` using `self.x`	Directly inside the class body, outside methods
Belongs to	Each **object** separately	The **class** — shared by all objects
Access	`obj.x`	`ClassName.x` or `obj.x`
Use	Data that differs (name, marks)	Common data (college name, counters)

```
class Student:
    college = "JNTUA"            # class attribute (shared)
    count = 0
    def __init__(self, name):
        self.name = name         # instance attribute (separate)
        Student.count += 1

a = Student("Asha"); b = Student("Ravi")
print(a.college, b.college, Student.count)
Student.college = "JNTU"         # changing it changes it for all objects
print(a.college)
```

Output

```
JNTUA JNTUA 2
JNTU
```

11. Encapsulation

Encapsulation means **wrapping data and the methods that use it into one unit (class)** and **hiding the data** from direct outside access, so it can only be changed in a controlled way. Python uses naming conventions for access levels:

Level	Syntax	Meaning
Public	`self.name`	Accessible from anywhere
Protected	`self._name`	Single underscore — a hint: “for use inside the class and subclasses”
Private	`self.__name`	Double underscore — name is changed internally (*name mangling*), so it cannot be accessed directly from outside

```
class Account:
    def __init__(self, balance):
        self.__balance = balance          # private attribute

    def deposit(self, amt):               # controlled access (setter-like)
        if amt > 0:
            self.__balance += amt

    def get_balance(self):                # getter
        return self.__balance

acc = Account(1000)
```

```
acc.deposit(500)
print(acc.get_balance())
print(acc.__balance)                    # direct access fails
```

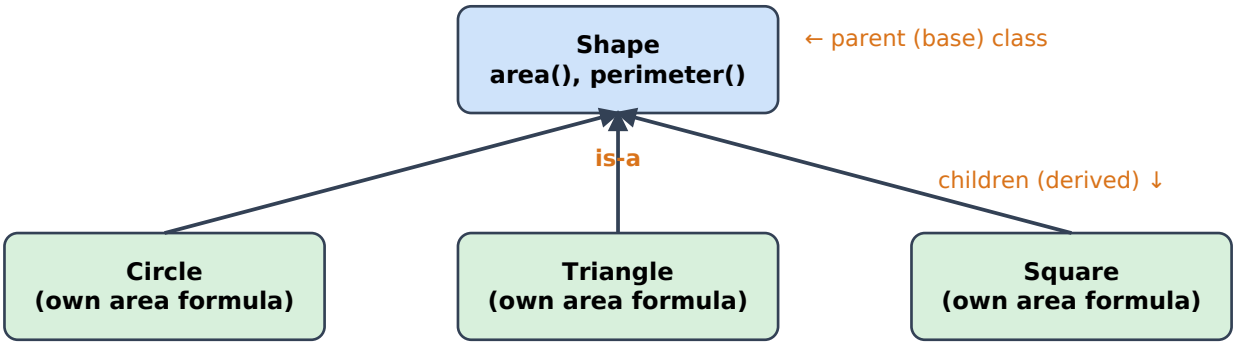
Output

```
1500
AttributeError: 'Account' object has no attribute '__balance'
```

> **i Remember**
>
> - Benefit: data protection, validation (reject negative deposits) and easier maintenance.

12. Inheritance

Inheritance lets a new class (**child / subclass / derived class**) acquire the attributes and methods of an existing class (**parent / superclass / base class**). It gives **code reuse** — write common code once in the parent.



Type	Meaning	Syntax
Single	One parent → one child	`class B(A):`
Multilevel	Grandparent → parent → child	`class C(B): (B inherits A)`
Multiple	Child with two or more parents	`class C(A, B):`
Hierarchical	One parent → many children	`class B(A) class C(A)`

```
class Person:
    def __init__(self, name):
        self.name = name
    def intro(self):
        print("Hi, I am", self.name)

class Student(Person):                  # single inheritance
    def __init__(self, name, roll):
        super().__init__(name)          # call parent constructor
        self.roll = roll
    def show(self):
        print("Roll no:", self.roll)

s = Student("Asha", 101)
s.intro()                               # inherited method
s.show()
print(isinstance(s, Person), issubclass(Student, Person))
```

Output

```
Hi, I am Asha
Roll no: 101
True True
```

13. Polymorphism

Polymorphism = “many forms”. The **same method name or operator behaves differently** for different objects.

- **Method overriding** — a child class redefines a parent’s method with the same name.
- **Duck typing** — if an object has the required method, Python does not care about its class.
- **Operator overloading** — special methods like *__add__* change the meaning of + for objects.

```
class Animal:
    def sound(self): print("Some sound")
class Dog(Animal):
    def sound(self): print("Bow Bow")      # overriding
class Cat(Animal):
    def sound(self): print("Meow")

for a in (Animal(), Dog(), Cat()):
    a.sound()                             # same call, different behaviour

class Point:
    def __init__(self, x): self.x = x
    def __add__(self, o): return Point(self.x + o.x)   # operator overloading
print((Point(3) + Point(4)).x)
```

Output

```
Some sound
Bow Bow
Meow
7
```

✓ Quick Revision

- File cycle: **open → read/write → close**; use **with** for auto-close. Modes: r, w (erase), a (append), r+, rb/wb.
- Reading: read(), readline(), readlines(). Writing: write(), writelines(). seek()/tell() move/show pointer.
- **pickle**: dump() / load() with binary modes. **csv**: reader, writer, DictReader, DictWriter.
- **os**: getcwd, listdir, mkdir, remove, rename · **os.path**: exists, join, isfile, splitext, getsize.
- **Class** = blueprint, **object** = instance, **self** = current object, **__init__** = constructor.
- Instance attribute (per object) vs class attribute (shared). Private data → **__name**.
- **Inheritance** → code reuse (super()). **Polymorphism** → one name, many forms (overriding).

SAMPLE PROGRAMS 18 – 23 (with output)

Program 18

Question

- Write a program to sort words in a file and put them in another file. The output file should have only lower-case words, so any upper-case words from source must be lowered.

Input file: `source.txt`

```
Python is Easy and Python is Powerful
Files are Fun and Useful
```

Program

```
# Read words, convert to lower case, sort and write to another file
with open("source.txt", "r") as f:
    words = f.read().split()            # split on spaces and newlines

words = [w.lower() for w in words]      # every word -> lower case
words.sort()                            # alphabetical order

with open("sorted.txt", "w") as out:
    for w in words:
        out.write(w + "\n")

print("Total words written:", len(words))
```

Output

```
Total words written: 12
```

Content of the output file `sorted.txt`:

```
and
and
are
easy
files
fun
is
is
powerful
python
python
useful
```

Logic: read → split into words → list comprehension with *lower()* → *sort()* → write each word on a new line.

Program 19

Question

- Write a Python program to print each line of a file in reverse order.

Input file: `source.txt`

```
Python is Easy and Python is Powerful
Files are Fun and Useful
```

Program

```
# Method 1: print the lines from last to first
with open("source.txt") as f:
    lines = f.readlines()

print("Lines in reverse order:")
for line in reversed(lines):
    print(line.rstrip("\n"))

# Method 2: reverse the characters of every line
print("\nEach line reversed:")
for line in lines:
    print(line.rstrip("\n")[::-1])
```

Output

```
Lines in reverse order:
Files are Fun and Useful
Python is Easy and Python is Powerful

Each line reversed:
lufrewoP si nohtyP dna ysaE si nohtyP
lufesU dna nuF era seliF
```

i Remember

- If your question means *“reverse the order of lines”* use Method 1; if it means *“reverse each line’s text”* use Method 2. Writing both is safe.

Program 20

Question

- Write a Python program to compute the number of characters, words and lines in a file.

Input file: `source.txt`

```
Python is Easy and Python is Powerful
Files are Fun and Useful
```

Program

```
chars = words = lines = 0
with open("source.txt") as f:
    for line in f:                      # one line at a time
        lines += 1
        words += len(line.split())
        chars += len(line)              # includes the newline character

print("Lines     :", lines)
```

```
print("Words     :", words)
print("Characters:", chars)
```

Output

```
Lines     : 2
Words     : 12
Characters: 63
```

i Remember

- Characters here include the newline (\n) at the end of the line. Use *len(line.rstrip("\n"))* to ignore it.

Program 21

Question

- Write a program to create, display, append, insert and reverse the order of the items in the array.

Program

```
from array import array

arr = array("i", [10, 20, 30, 40])      # create an integer array
print("Created  :", arr.tolist())

print("Display  :", end=" ")
for x in arr:                           # display items
    print(x, end=" ")
print()

arr.append(50)                          # add at the end
print("Appended :", arr.tolist())

arr.insert(2, 25)                       # insert 25 at index 2
print("Inserted :", arr.tolist())

arr.reverse()                           # reverse the order
print("Reversed :", arr.tolist())
```

Output

```
Created  : [10, 20, 30, 40]
Display  : 10 20 30 40
Appended : [10, 20, 30, 40, 50]
Inserted : [10, 20, 25, 30, 40, 50]
Reversed : [50, 40, 30, 25, 20, 10]
```

i Remember

- array("i") stores integers of one type only (type code **i** = int, **f** = float, **d** = double). Other methods: *remove(), pop(), index(), count()*.

Program 22

Question

- Write a program to add, transpose and multiply two matrices.

Program

```
A = [[1, 2, 3],
     [4, 5, 6]]
B = [[7, 8, 9],
     [10, 11, 12]]

# Addition (same order matrices)
add = [[A[i][j] + B[i][j] for j in range(3)] for i in range(2)]
print("A + B =", add)

# Transpose of A (rows become columns)
trans = [[A[i][j] for i in range(2)] for j in range(3)]
print("Transpose of A =", trans)

# Multiplication: A (2x3) x Bt (3x2) -> 2x2
Bt = [[B[i][j] for i in range(2)] for j in range(3)]
mul = [[0, 0], [0, 0]]
for i in range(2):
    for j in range(2):
        for k in range(3):
            mul[i][j] += A[i][k] * Bt[k][j]
print("A x B^T =", mul)
```

Output

```
A + B = [[8, 10, 12], [14, 16, 18]]
Transpose of A = [[1, 4], [2, 5], [3, 6]]
A x B^T = [[50, 68], [122, 167]]
```

✓ Quick Revision

- Addition needs **same order**. Multiplication needs **columns of first = rows of second**; result order = rows of first × columns of second.

Program 23

Question

- Write a Python program to create a class that represents a shape. Include methods to calculate its area and perimeter. Implement subclasses for different shapes like circle, triangle, and square.

Program

```
import math

class Shape:                                    # parent class
    def area(self):
        return 0
    def perimeter(self):
        return 0

class Circle(Shape):
    def __init__(self, r):
        self.r = r
    def area(self):
```

```
        return math.pi * self.r ** 2
    def perimeter(self):
        return 2 * math.pi * self.r

class Triangle(Shape):
    def __init__(self, a, b, c):
        self.a, self.b, self.c = a, b, c
    def area(self):                          # Heron's formula
        s = (self.a + self.b + self.c) / 2
        return math.sqrt(s * (s - self.a) * (s - self.b) * (s - self.c))
    def perimeter(self):
        return self.a + self.b + self.c

class Square(Shape):
    def __init__(self, side):
        self.side = side
    def area(self):
        return self.side ** 2
    def perimeter(self):
        return 4 * self.side

for s in (Circle(5), Triangle(3, 4, 5), Square(4)):
    print(f"{type(s).__name__:<9} Area = {s.area():7.2f}   Perimeter = {s.perimeter():6.2f}")
```

Output

```
Circle    Area =   78.54   Perimeter =  31.42
Triangle  Area =    6.00   Perimeter =  12.00
Square    Area =   16.00   Perimeter =  16.00
```

i Remember

- Concepts used: **inheritance** (subclasses of Shape), **method overriding** and **polymorphism** (same *area()* call works for every shape).

Possible Exam Questions

Short answer (2–3 marks)	Long answer (5–10 marks)
1. Differentiate text and binary files. 2. List any four file opening modes. 3. What is the use of the *with* statement? 4. What is pickling? 5. What is a constructor? 6. Differentiate class and instance attributes. 7. What is encapsulation? 8. What is method overriding?	1. Explain the methods to read and write text files with examples. 2. Explain the *pickle* and *csv* modules with programs. 3. Explain *os* and *os.path* functions with examples. 4. Explain classes, objects and constructors with a program. 5. Explain the types of inheritance with examples. 6. Explain encapsulation and polymorphism with programs. 7. Write programs 18–23 from the sample experiments.